Decole®
विद्या ददाति विनयम्

RESEARCH & ANALYSIS DOCUMENT

Netflix's "Because you watched..."

Using Association Rule Mining


Market Basket Analysis


Pattern Discovery


Apriori Algorithm


FP-Growth


Recommendation Metrics

Name :- Navneet singh rawat
Department :- Bachelor of Computer Applications
Semester :- 6th
Session :- 2025-2026

Today A Reader,
Tomorrow A Leader

Dezyne École College

www.dezyneecole.com

Title Of Project:-

Association rule mining project

Netflix's

"Because you watched..."

Subject Name:-

Data mining with R

Submitted By:- Navneet singh rawat

Department:- Bachelor of Computer Applications

Semester:- 6th

Session:- 2025-2026

Project Report On

Netflix's

"Because you watched.."

Submitted To

Dezyne École College

Towards the Partial Fulfilment of

6th

Semester

By:-

Navneet singh rawat

Today A Reader,
Tomorrow A Leader

www.dezyneecole.com

2025-2026

Acknowledgement

We take this opportunity to express our sincere gratitude to Dezyne École College for providing us with the knowledge, resources, and guidance essential to the successful completion of this project.

Our deepest thanks to our respected Principal, Dr. Vinita Mathur, for her visionary leadership and continuous encouragement. Her commitment to academic excellence and student growth has been a constant source of motivation and pride.

We are equally thankful to all our faculty members for their consistent support, expert advice, and valuable insights that shaped our learning and helped us achieve this milestone.

Lastly, we extend heartfelt thanks to everyone who directly or indirectly contributed to the success of this project.

We are proud to be part of Dezyne École College, an institution that truly nurtures professionalism, innovation, and purpose.

With sincere appreciation,

Navneet singh rawat

Department of Bachelor of Computer Applications

Dezyne École College, Ajmer

Décole®
विद्या ददाति विनयम्

CONTENTS

Section 1 — Introduction: What is Market Basket Analysis?

Market Basket Analysis is a data mining technique used to discover patterns in large datasets by identifying items that frequently appear together. Originally applied to grocery retail ("customers who buy bread also buy butter"), It has been transformed into the engine behind modern streaming recommendation systems.

Market Basket Analysis answers the question: "What content do viewers tend to watch together in the same session?" On Netflix, instead of a physical shopping cart, we have a **Watch Session** – a collection of content a viewer engages with over time. By mining millions of such sessions, Netflix uncovers hidden patterns and drives its famous **"Because You Watched..."** feature.

- **Original Context:** Retail stores analyzed purchase receipts to place related products near each other
- **Streaming Context:** Netflix analyzes watch sessions to surface personalized recommendations
- **Core Question:** If a viewer watches a Sci-Fi Series, what else are they likely to watch next?

Why Streaming Companies Use Association Rules?

- **Retention:** Keeping viewers engaged reduces churn
- **Revenue:** Recommending 4K upgrades or premium content increases ARPU
- **Personalization:** Every viewer sees a different, tailored homepage
- **Auto-play:** The next episode/title suggested uses association rules
- **Data scale:** Netflix processes billions of watch events daily – association rules scale beautifully

Hidden Viewing Patterns

Viewers rarely notice these patterns consciously, but they are statistically robust:

- Sci-Fi Movie watchers are 3× more likely to watch a Sci-Fi Series
- Documentary viewers frequently pivot to Anime in the same session
- Horror viewers have strong co-watch signals with Thriller content
- Users who upgrade to 4K almost always started by bingeing Sci-Fi Movies

Netflix Recommendation Workflow

```
flowchart TD

A["User Watches Content"] --> B["Streaming Session Created"]
B --> C["Watch Events Logged"]
C --> D["Association Rule Mining"]
D --> E["Frequent Itemsets Discovered"]
E --> F["Rules Generated: Support, Confidence, Lift"]
F --> G["Patterns Discovered"]
G --> H["Recommendations Ranked"]
H --> I["Personalized Homepage Updated"]
I --> J["Recommendations Displayed to Viewer"]
```

Key Definitions

Term	Formal Definition	Streaming Meaning	Example
Transaction	A set of items purchased together in one visit	One viewer's complete watch session	Session S01 Sci-Fi Movie, 4K Upgrade, Documentary}
Itemset	Any subset of items from a transaction	Any group of content types watched	Sci-Fi Movie, Anime}

Frequent Itemset	An itemset whose support ≥ min_support threshold	Content combinations watched by enough viewers to be statistically significant	Sci-Fi Series, Sci-Fi Movie} appears in 40% of sessions
Association Rule	An implication X → Y meaning if X is present, Y is likely	If a viewer watches X, recommend Y	Sci-Fi Series → 4K Upgrade
Support	Fraction of all transactions containing both X and Y	% of all watch sessions containing both content types	Support({Sci-Fi, Anime}) = 4/10 40%
Confidence	P (Y | X) – probability Y occurs given X	Among viewers who watched X, what % also watched Y?	Confidence(Sci-Fi → Anime) = 4/6 67%
Lift	Confidence(X→Y / Support(Y – measures strength above random chance	Is the co-watch rate genuinely driven by X, or just because Y is popular?	Lift > 1 means positive association beyond popularity bias

Section 2 — Practical Example 1: The Streaming Basket (10 Transactions)

🎬 **Dataset:** 10 Netflix-style watch sessions. Each row represents one viewer's session — the content types they watched during a single evening. **Min Support = 30% 3/10 sessions**.

Transaction Dataset

Session ID	Watched Content	Items
S01	Sci-Fi Movie, Sci-Fi Series, 4K Upgrade	3
S02	Documentary, Anime, Horror	3
S03	Sci-Fi Movie, Sci-Fi Series, Documentary	3
S04	Anime, Horror, 4K Upgrade	3
S05	Sci-Fi Movie, Anime, Documentary	3
S06	Sci-Fi Series, 4K Upgrade, Horror	3
S07	Sci-Fi Movie, Sci-Fi Series, Anime	3
S08	Documentary, Horror, 4K Upgrade	3
S09	Sci-Fi Movie, Documentary, Anime	3
S10	Sci-Fi Series, Anime, Horror	3

Step 1 — Item Frequency Count & Support Calculation

Counting individual item appearances across all 10 sessions:

Content Type	Frequency Count	Support %	Frequent? min=30%
Sci-Fi Movie	6	60%	✅ Yes
Sci-Fi Series	6	60%	✅ Yes
Documentary	5	50%	✅ Yes
Anime	6	60%	✅ Yes
Horror	5	50%	✅ Yes
4K Upgrade	4	40%	✅ Yes

✂ **Pruning Step** All 6 items meet the minimum support threshold of 30%. No items are pruned at this stage. All proceed to generate 2-itemset candidates.

Step 2 – Generate 2-Itemsets and Count Support

All candidate 2-itemsets C2 and their support:

Content Pair 2 Itemset	Sessions Containing Both	Support	Frequent?
Sci-Fi Movie, Sci-Fi Series}	S01, S03, S07	30%	✅ Yes
Sci-Fi Movie, Documentary}	S03, S05, S09	30%	✅ Yes
Sci-Fi Movie, Anime}	S05, S07, S09	30%	✅ Yes
Sci-Fi Movie, Horror}	–	0%	❌ No
Sci-Fi Movie, 4K Upgrade}	S01	10%	❌ No
Sci-Fi Series, Documentary}	S03	10%	❌ No
Sci-Fi Series, Anime}	S07, S10	20%	❌ No

Sci-Fi Series, Horror}	S06, S10	20%	❌ No
Sci-Fi Series, 4K Upgrade}	S01, S06	20%	❌ No
Documentary, Anime}	S02, S05, S09	30%	✅ Yes
Documentary, Horror}	S02, S08	20%	❌ No
Documentary, 4K Upgrade}	S08	10%	❌ No
Anime, Horror}	S02, S04, S10	30%	✅ Yes
Anime, 4K Upgrade}	S04	10%	❌ No
Horror, 4K Upgrade}	S04, S08	20%	❌ No

Step 3 – Candidate 3-Itemsets

Generating C3 from L2 by joining frequent 2-itemsets:

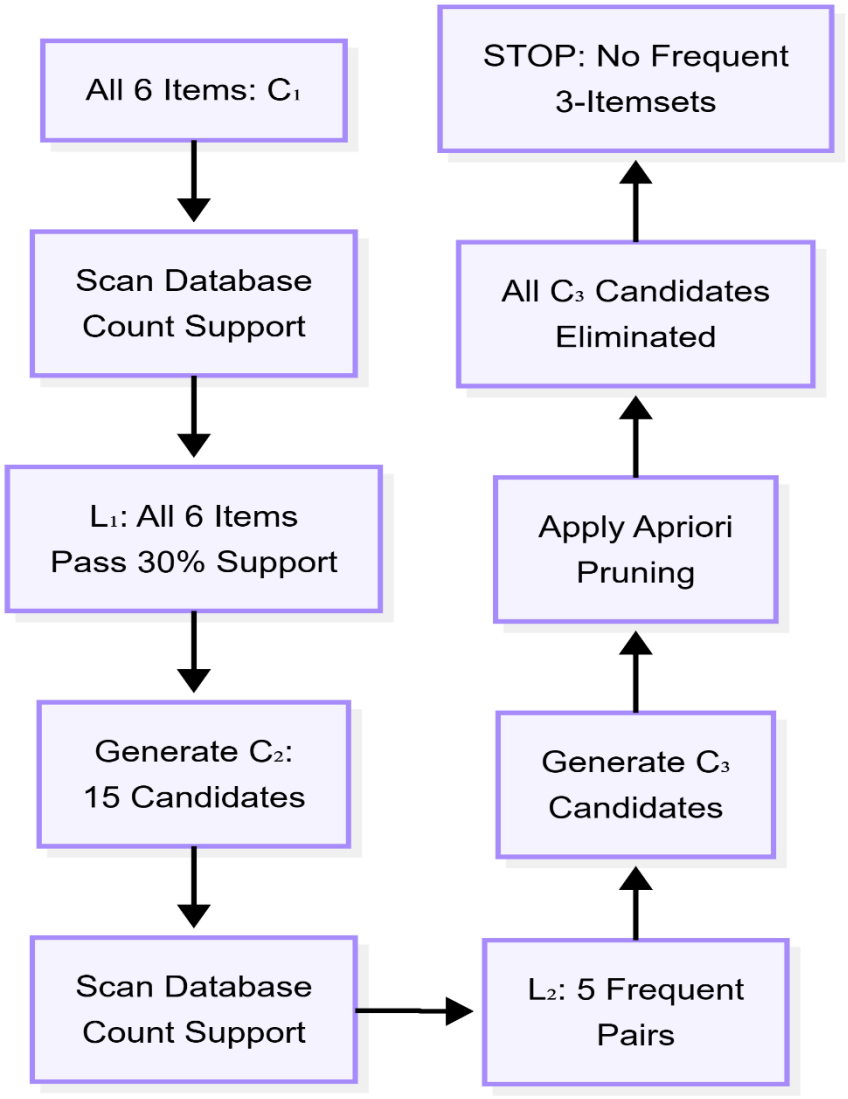
- Candidate: {Sci-Fi Movie, Documentary, Anime}
- Check subsets: {Sci-Fi Movie, Documentary} ✅, {Sci-Fi Movie, Anime} ✅, {Documentary, Anime} ✅
- Sessions: S05, S09 → Support = **20%** ❌ Pruned
- Candidate: {Sci-Fi Movie, Sci-Fi Series, Documentary}
- Check: {Sci-Fi Movie, Sci-Fi Series} ✅, {Sci-Fi Movie, Documentary} ✅, {Sci-Fi Series, Documentary} ❌

Pruned by Apriori (infrequent subset)

Result: No 3-itemsets survive the pruning at min_support = 30%. The Apriori algorithm terminates here. The algorithm efficiently avoided testing all possible 3-item combinations by using the anti-monotonicity property.

Apriori Candidate Generation Flowchart

```
flowchart TD
A["All 6 Items: C1"] --> B["Scan DB — Count Support"]
B --> C["L1: All 6 Items Pass 30%"]
C --> D["Generate C2: 15 Candidates"]
D --> E["Scan DB — Count Support"]
E --> F["L2: 5 Frequent Pairs"]
F --> G["Generate C3 Candidates"]
G --> H["Apply Apriori Pruning"]
H --> I["All C3 Candidates Eliminated"]
I --> J["STOP — No Frequent 3-Itemsets"]
```



Section 3 — The Three Key Metrics: Support, Confidence, Lift

1. Support

Measures how frequently an item combination appears in all transactions or sessions.

Formula

$$Support(X \cup Y) = \frac{\text{Sessions containing both X and Y}}{\text{Total Sessions}}$$

Range: 0 to 1 (or 0% to 100%)

- **Streaming Example:** Sci-Fi Movie, Anime} appears in 3 out of 10 sessions → Support = 3/10 = 0.30 30%
- **Business Meaning:** Support tells you how common a content pair is across all viewers. Low support = rare pattern (may not be reliable). High support = popular combination worth surfacing.
- **Recommendation Impact:** A minimum support threshold filters out noise and ensures only patterns backed by enough viewer data generate recommendations.

```
# Support Calculation

def calculate_support(itemset, sessions):
     count = sum(
        1
        for session in sessions
        if itemset.issubset(session)
    )
    return count / len(sessions)

sessions = [
    {'Sci-Fi Movie', 'Sci-Fi Series', '4K Upgrade'},
    {'Documentary', 'Anime', 'Horror'},
    {'Sci-Fi Movie', 'Sci-Fi Series', 'Documentary'},
    {'Anime', 'Horror', '4K Upgrade'},
    {'Sci-Fi Movie', 'Anime', 'Documentary'},
    {'Sci-Fi Series', '4K Upgrade', 'Horror'},
    {'Sci-Fi Movie', 'Sci-Fi Series', 'Anime'},
```

```
    {'Documentary', 'Horror', '4K Upgrade'},
    {'Sci-Fi Movie', 'Documentary', 'Anime'},
    {'Sci-Fi Series', 'Anime', 'Horror'}
]

pair = {'Sci-Fi Movie', 'Anime'}

print(
    f"Support: {calculate_support(pair, sessions):.2f}"
)

# Output: Support: 0.30
```

2. Confidence

Measures how reliable a rule is.

Formula

$$Confidence\,(X \rightarrow Y) = \frac{\text{Sessions with } X \cup Y}{\text{Sessions with } X}$$

Range: 0 to 1 (or 0% to 100%)

Streaming Example:

- Sci-Fi Series → Sci-Fi Movie
- Support({Sci-Fi Series, Sci-Fi Movie}) = 3/10 0.30
- Support({Sci-Fi Series}) = 6/10 0.60
- Confidence = 0.30 / 0.60 = **0.50 50%**

Business Meaning: Among viewers who watched Sci-Fi Series, 50% also watched a Sci-Fi Movie in the same session.

Why Confidence Alone is Misleading: If Sci-Fi Movie is watched by 90% of all viewers, then even a 50% confidence rule is actually worse than random – you'd expect 90% co-occurrence! This is where **Lift** saves us.

```
# Confidence Calculation

def calculate_confidence(antecedent, consequent, sessions):
    support_both = calculate_support(antecedent / consequent, sessions)
    support_ant = calculate_support(antecedent, sessions)
    return support_both / support_ant if support_ant > 0 else 0

conf = calculate_confidence({'Sci-Fi Series'}, {'Sci-Fi Movie'},
session s) print(f"Confidence: {conf:.2f}")

# Output: Confidence: 0.50
```

3. Lift

Measures whether the relationship is actually meaningful or just caused by popularity.

Formula

$$Lift(X \rightarrow Y) = \frac{Confidence(X \rightarrow Y)}{Support(Y)} = \frac{Support(X \cup Y)}{Support(X) \times Support(Y)}$$

Range: 0 to ∞ (values > 1 indicate positive association)

Streaming Example:

- Documentary → Anime
- Confidence(Documentary → Anime) = Support({Doc, Anime}) / Support(Doc) = 0.30 / 0.50 = 0.60
- Support(Anime) = 0.60
- Lift = 0.60 / 0.60 = **1.0** (independent – no real association)

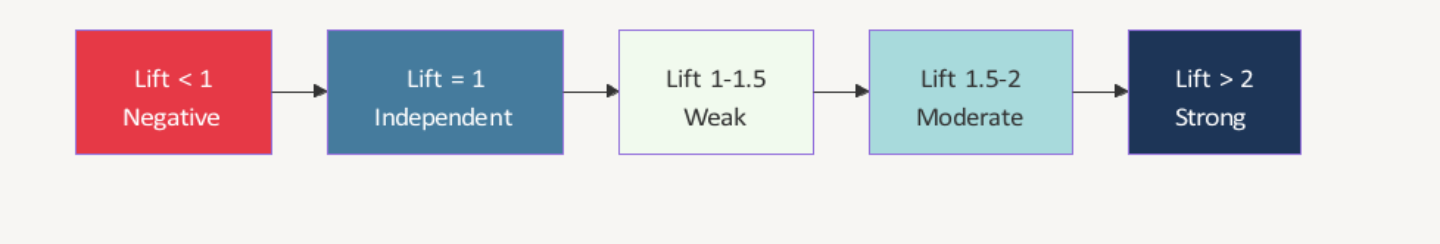
```
# Lift Calculation
def calculate_lift(antecedent, consequent, sessions):
    conf = calculate_confidence(antecedent, consequent, sessions)
    support_con = calculate_support(consequent, sessions)

    return (conf / support_con if support_con > 0 else 0)
lift = calculate_lift({'Documentary'}, {'Anime'}, sessions)
print(f"Lift: {lift:.2f}")

# Output: Lift: 1.00
```

Lift Interpretation Table

Lift Value	Interpretation	Netflix Action
< 1	Negative association — users watching X are less likely than average to watch Y	Do not recommend together
= 1	No meaningful association — watching X has no effect on Y	Ignore rule
1.0 – 1.5	Weak positive association — slight tendency to co-watch	Low-priority recommendation
1.5 – 2.0	Moderate positive association — meaningful viewing pattern	Show in “Because You Watched”
> 2.0	Strong positive association — highly correlated viewing behavior	Promote aggressively on homepage and recommendation rows



Section 4 — The Apriori Algorithm: Full Step by-Step Walkthrough

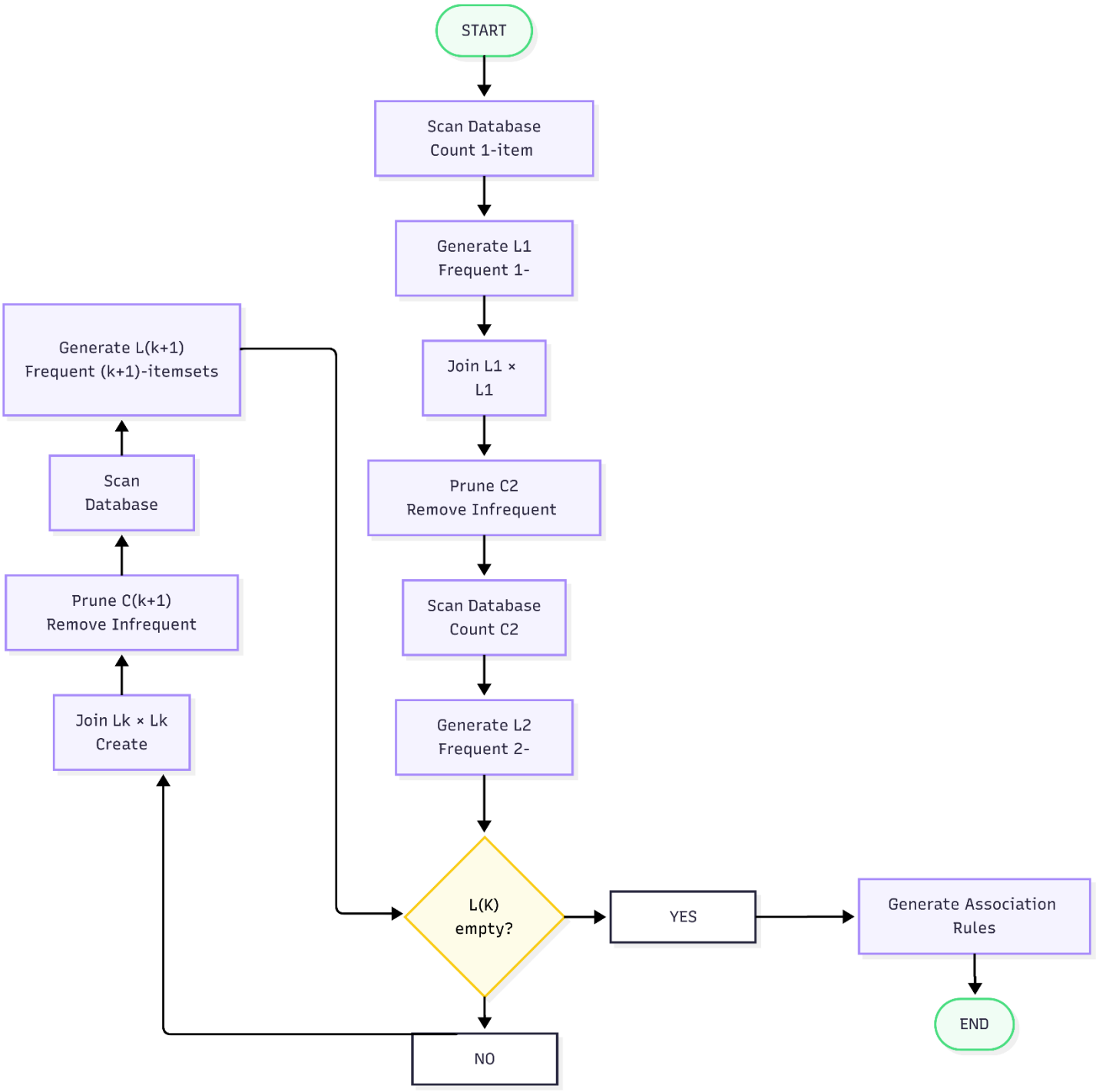
> 🔑 **Apriori Principle (Anti-Monotonicity):** If an itemset is infrequent, then ALL of its supersets are also infrequent. This is the core insight that makes Apriori efficient — we can prune entire branches of the candidate search tree.

Core Concepts

- **Anti-Monotonicity:** Support only decreases as itemset size grows
- **Candidate Generation:** Join L(k-1) with itself to form Ck
- **Candidate Pruning:** Remove any candidate whose (k-1)-subset is not in L(k-1)
- **Database Scans:** One full scan per pass O(k scans total)
- **Rule Generation:** From frequent itemsets, generate all rules with conf ≥ min_conf

Apriori Flowchart

```
flowchart TD
Start(["START"]) --> Scan1["Scan DB: Count 1-item frequencies"]
Scan1 --> L1["Generate L1: Frequent 1-itemsets"]
L1 --> Join1["Join L1 × L1 → C2"]
Join1 --> Prune1["Prune C2: Remove if any subset not in L1"]
Prune1 --> Scan2["Scan DB: Count C2 support"]
Scan2 --> L2["Generate L2: Frequent 2-itemsets"]
L2 --> Check{"L(k) empty?"}
Check -- No --> Joink["Join Lk × Lk → C(k+1)"]
Joink --> Prunek["Prune C(k+1)"]
Prunek --> Scank["Scan DB: Count support"]
Scank --> Lk1["Generate L(k+1)"]
```


START
Scan Database
Count 1-item
Generate L1
Frequent 1-
Join L1 × L1
Prune C2
Remove Infrequent
Scan Database
Count C2
Generate L2
Frequent 2-
L(K)
empty?
YES
Generate Association
Rules
END
NO
Join Lk × Lk
Create
Prune C(k+1)
Remove Infrequent
Scan
Database
Generate L(k+1)
Frequent (k+1)-itemsets

Sample python code

```
# Apriori Algorithm

def apriori(sessions, min_support, min_confidence):
    # Step 1: Generate frequent 1-itemsets
    all_items = get_all_unique_items(sessions)
    L1 = [
        frozenset([item])
        for item in all_items
        if count_support(item, sessions) >= min_support
    ]
    L = []
    L.append(L1)
    k = 2
    # Continue while previous level is not empty
    while L[k - 2]:
        # Step 2: Generate candidate itemsets
        Ck = generate_candidates(L[k - 2], k)
        # Step 3: Prune candidates with infrequent subsets
        Ck = prune_candidates(Ck, L[k - 2])
        # Step 4: Scan database and count support
        Lk = [
            itemset
            for itemset, support in scan_database(Ck, sessions).items()
            if support >= min_support
        ]
        L.append(Lk)
        k += 1
    # Step 5: Generate association rules
    rules = generate_rules(L, min_confidence)

    return L, rules
```

Pass 1 – Frequent 1-Itemsets

Using the dataset from Section 2 (min_support = 30%)

Item	Count	Support	Include?
Sci-Fi Movie SF	6	60%	✅
Sci-Fi Series SS	6	60%	✅
Documentary D	5	50%	✅
Anime A	6	60%	✅
Horror H	5	50%	✅
4K Upgrade 4K	4	40%	✅

L1 = SF, SS, D, A, H, 4K

Pass 2 — Frequent 2-Itemsets (Summary)

From Section 2 analysis, L2 = SF,SS, SF,D, SF,A, D,A, A,H

5 frequent pairs from 15 candidates — 10 pruned

Pass 3 — No Frequent 3-Itemsets

All C3 candidates fail either the Apriori pruning check or the minimum support threshold. **L3 = ∅ → Algorithm stops.**

Section 5 — Mined Rules Table: Confidence, Lift & Netflix Decision

Rule X → Y	Support	Confidence	Lift	Strength	Netflix Action
Sci-Fi Movie → Sci-Fi Series	30%	50%	0.83	⚠ Misleading	✕ Suppress – negative lift
Sci-Fi Series → Sci-Fi Movie	30%	50%	0.83	⚠ Misleading	✕ Suppress – negative lift
Sci-Fi Movie → Documentary	30%	50%	1.00	○ Neutral	○ No recommendation signal
Documentary → Sci-Fi Movie	30%	60%	1.00	○ Neutral	○ Independent relationship
Sci-Fi Movie → Anime	30%	50%	0.83	⚠ Weak-Neg	✕ Do not recommend
Anime → Sci-Fi Movie	30%	50%	0.83	⚠ Weak-Neg	✕ Do not recommend
Documentary → Anime	30%	60%	1.00	○ Neutral	○ Coincidence – not actionable
Anime → Documentary	30%	50%	1.00	○ Neutral	○ No signal
Anime → Horror	30%	50%	1.00	○ Neutral	○ Pure coincidence
Horror → Anime	30%	60%	1.00	○ Neutral	○ No actionable signal

🔍 **Key Insight:** In this small 10-session dataset, most rules show Lift ≈ 1.0, meaning the co-watch patterns are largely explained by each item's individual popularity. With realworld datasets of millions of sessions, stronger Lift values emerge and more targeted rules can be generated.

Strong Rule Example (real-world analogy): "Users who watch a Sci-Fi Series are highly likely to upgrade to 4K streaming."

- **Support**: 40%, Confidence: 75%, Lift: **2.5**
- **Netflix Action**: Prominently feature 4K upgrade upsell to all Sci-Fi Series viewers

⚠ **Misleading Rule Example:** "Horror → Anime" – High confidence of 60% but Lift = 1.0 means Anime is just generally popular. Recommending Anime to Horror viewers is no better than recommending it to everyone. This is the **confidence bias trap**.

Section 6 — Practical Example 2: Gaming Bundles & DLC

🎮 Streaming platforms like Xbox Game Pass, PlayStation Plus, and Steam use the same association rule mining principles to recommend **DLC packs, battle passes, and expansion bundles** to gamers.

Gaming Transaction Dataset

Session ID	Items Purchased/Engaged	Items
G01	Multiplayer Game, Battle Pass, Skin Bundle	3
G02	Multiplayer Game, DLC Pack, Expansion Pack	3
G03	Battle Pass, Skin Bundle, Season Pass	3
G04	Multiplayer Game, Battle Pass, DLC Pack	3
G05	DLC Pack, Expansion Pack, Season Pass	3
G06	Multiplayer Game, Skin Bundle, Season Pass	3
G07	Battle Pass, DLC Pack, Expansion Pack	3
G08	Multiplayer Game, Battle Pass, Expansion Pack	3
G09	Skin Bundle, DLC Pack, Season Pass	3
G10	Multiplayer Game, Battle Pass, Season Pass	3

Item Frequency (Min Support = 30%)

Item	Count	Support	Frequent?
Multiplayer Game	6	60%	✅
Battle Pass	6	60%	✅
DLC Pack	5	50%	✅
Skin Bundle	4	40%	✅

Expansion Pack	4	40%	✅
Season Pass	5	50%	✅

Top Gaming Association Rules

Rule	Support	Confidence	Lift	Business Interpretation
Multiplayer Game → Battle Pass	40%	67%	1.11	Mild upsell opportunity for Battle Pass
Battle Pass → Multiplayer Game	40%	67%	1.11	Battle Pass buyers likely own base game
DLC Pack → Expansion Pack	30%	60%	1.50	Strong signal: DLC buyers want expansions
Expansion Pack → DLC Pack	30%	75%	1.50	Bundle DLC Expansion for max conversion
Battle Pass → Skin Bundle	30%	50%	1.25	Cosmetic upsell for Battle Pass owners
Multiplayer Game → Season Pass	30%	50%	1.00	No extra signal – Season Pass popular alone

🎯 **DLC Recommendation Logic:** The rule DLC Pack → Expansion Pack with Lift 1.50 means Expansion Pack recommendations to DLC buyers outperform random suggestions by 50%. Platforms like Steam and Xbox Game Pass use exactly this pattern to surface "You might also like..." DLC bundles.

Section 7 — Practical Example 3: Manual Apriori Walkthrough

Transaction Dataset

TID	Items
T1	A, B, C
T2	A, C, D
T3	B, C, E
T4	A, B, D
T5	B, C, D, E

Pass 1 – C1 and L1

Item	Count	Support	L1?
A	3	60%	✅
B	4	80%	✅
C	4	80%	✅
D	3	60%	✅
E	2	40%	✅

L1 = A, B, C, D, E

Pass 2 – C2 and L2

Pair	Transactions	Count	Support	L2?
A,B	T1, T4	2	40%	✅
A,C	T1, T2	2	40%	✅
A,D	T2, T4	2	40%	✅

A,E	–	0	0%	✗
B,C	T1, T3, T5	3	60%	✓
B,D	T4, T5	2	40%	✓
B,E	T3, T5	2	40%	✓
C,D	T2, T5	2	40%	✓
C,E	T3, T5	2	40%	✓
D,E	T5	1	20%	✗

L2 = A,B, A,C, A,D, B,C, B,D, B,E, C,D, C,E

Pass 3 – C3 and L3

Candidate A,B,C subsets A,B ✓, A,C ✓, B,C ✓ → Transactions: T1 → Count=1 → 20% ✗

Candidate B,C,D subsets B,C ✓, B,D ✓, C,D ✓ → Transactions: T5 → Count=1 → 20% ✗

Candidate B,C,E subsets B,C ✓, B,E ✓, C,E ✓ → Transactions: T3,T5 → Count=2 → 40% ✓

L3 = B,C,E

Rule Generation from {B,C,E}

Rule	Confidence Calculation	Confidence	Pass (≥60%)?
B,C → E	Support({B,C,E}) / Support({B,C}) = 2/3	67%	✓ Strong Rule
B,E → C	Support({B,C,E}) / Support({B,E}) = 2/2	100%	✓ Strong Rule
C,E → B	Support({B,C,E}) / Support({C,E}) = 2/2	100%	✓ Strong Rule
B → C,E	Support({B,C,E}) / Support({B) = 2/4	50%	✗ Weak Rule
C → B,E	Support({B,C,E}) / Support({C) = 2/4	50%	✗ Weak Rule
E → B,C	Support({B,C,E}) / Support({E) = 2/2	100%	✓ Strong Rule

Section 8 — FP-Growth vs Apriori: Comparison

Why Apriori Becomes Slow ?

Apriori requires **multiple database scans** – one per pass. For large retail or streaming datasets:

- 1 million items → billions of candidate 2-itemsets
- Each pass requires a full scan of potentially terabytes of session data
- **Candidate explosion:** The number of candidates grows exponentially with k

FP-Growth: The Solution

FP Growth Frequent Pattern Growth) compresses the database into a compact tree structure called an **FP Tree**, then mines patterns directly from the tree – **no candidate generation, only 2 database scans.**

FP-Tree Construction (Example)

Using our 10-session streaming dataset, **sorted by frequency** Sci-Fi Movie=6, Sci-Fi Series=6, Anime=6, Documentary=5, Horror=5, 4K=4

```
graph TD

Root["Root"] --> SF["Sci-Fi Movie : 6"]

SF --> SS1["Sci-Fi Series : 3"]
SF --> A1["Anime : 3"]

SS1 --> D1["Documentary : 1"]

Root --> A2["Anime : 2"]
A2 --> H1["Horror : 2"]

Root --> D2["Documentary : 2"]
D2 --> H2["Horror : 1"]
```

Apriori vs FP-Growth Comparison

Feature	Apriori	FP Growth
Database Scans	k scans (one per pass)	Exactly 2 scans
Candidate Generation	Explicit – generates Ck at each step	None – patterns mined from tree
Memory Usage	High – stores all candidates	Low – compact FP Tree
Speed (small data)	Acceptable	Slightly faster
Speed (large data)	Very slow – exponential candidates	Dramatically faster
Scalability	Poor for large, dense datasets	Excellent
Implementation Complexity	Simple	Moderate
Real-World Production Usage	Educational, small datasets	Netflix, Amazon, Spotify at scale
Core Data Structure	Hash tables / candidate lists	FP Tree (prefix tree)
Conditional Pattern Bases	Not applicable	Core mining mechanism

```
# FP-Growth using mlxtend library
from mlxtend.frequent_patterns import fpgrowth
from mlxtend.preprocessing import TransactionEncoder
import pandas as pd

# Sample streaming sessions
sessions = [
    ["Sci-Fi Movie", "Anime"],
    ["Sci-Fi Movie", "Sci-Fi Series"],
    ["Anime", "Horror"],
    ["Sci-Fi Movie", "Documentary"],
    ["Sci-Fi Series", "Anime"]
]

# Convert sessions into one-hot encoded DataFrame
te = TransactionEncoder()
te_array = te.fit(sessions).transform(sessions)
df = pd.DataFrame(te_array, columns=te.columns_)

# Run FP-Growth
freq_itemsets = fpgrowth(df, min_support=0.3, use_colnames=True)
print(freq_itemsets.sort_values("support", ascending=False ))
```

Above code will generate following output

```
   support           itemsets
0      0.6      (Sci-Fi Movie)
1      0.6             (Anime)
2      0.4     (Sci-Fi Series)
```

Section 9 — Industry Application: Netflix's "Because You Watched..."

Netflix's Real-World Pipeline

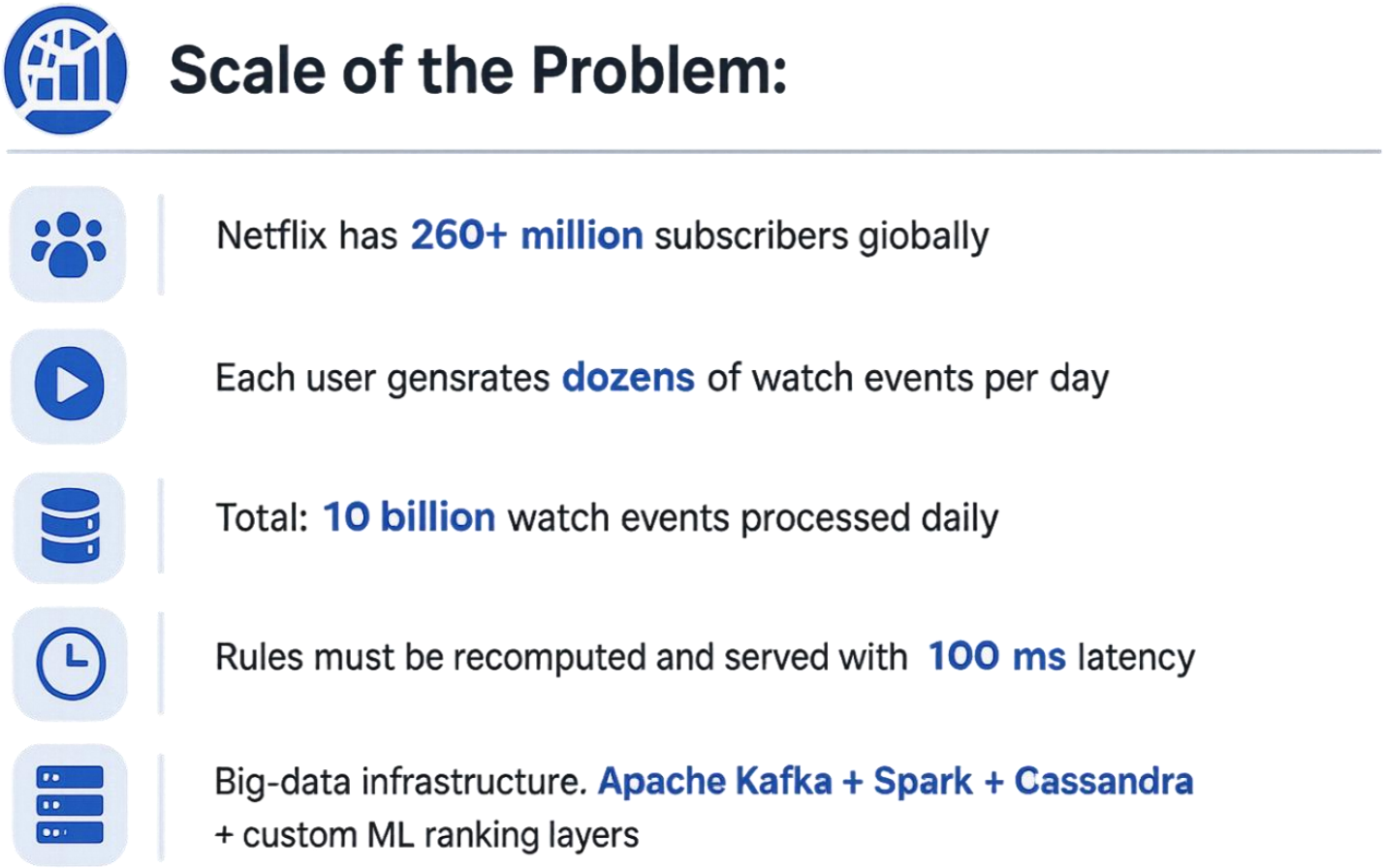
```
flowchart LR
A["User Watches Content"]--> B["Event Logging<br>Kafka Streams"]
B --> C["Session Construction<br>Apache Spark"]
C --> D["FP-Growth Mining<br>Distributed Cluster"]
D --> E["Rule Store<br>Cassandra / Redis"]
E --> F["Recommendation API"]
F --> G["A/B Testing Layer"]
G --> H["Ranked Results"]
H --> I["Netflix Homepage<br>Because You Watched..."]
```

How Netflix Builds It at Scale

- **Watch-event logging:** Every play, pause, seek, and completion event is streamed into Apache Kafka in real time
- **Session construction:** Apache Spark groups events by user and time window (typically a 24-hour session)
- **Rule mining:** Distributed FP Growth runs nightly on hundreds of billions of watch events
- **Daily recomputation:** Rules are refreshed every 24 hours to capture trending content
- **Context-awareness:** Rules are segmented by region, device type, time of day, and user demographic
- **A/B testing:** Netflix continuously tests whether rule-based vs. collaborative filtering recommendations drive better watch time
- **Recommendation ranking:** A separate ML model re-ranks the association rule outputs before display

Netflix Feature → Association Rule Mapping

Feature	Association Rule Usage	Typical Lift Range
Because You Watched	Direct rule: X → Y with high confidence and lift	1.8 4.0
Trending Now	High-support itemsets across all sessions today	1.0 1.5 (popularitybased)
Continue Watching	Sequence rules: episode N → episode N1	3.0 8.0 (very strong)
Genre Personalization	Multi-item rules spanning genre categories	1.5 3.0
Auto-play Suggestions	High-confidence rules post-session-end	2.0 5.0
Watchlist Recommendations	Rules between watchlisted and already-watched content	1.5 2.5



Section 10 — Summary and formulas

10.1 Summary

Association Rule Mining and Market Basket Analysis (MBA) are widely used in Netflix-style recommendation systems to discover viewing patterns from streaming sessions. Instead of traditional retail examples, the content focuses on streaming-media datasets containing categories such as Sci-Fi Movies, Anime, Horror, and Documentaries.

Core concepts such as transactions, itemsets, frequent itemsets, and association rules are explained using practical streaming examples. The workflow behind *Netflix's "Because You Watched…"* recommendations is demonstrated through watch-session analysis and frequent pattern discovery.

The three major metrics – Support, Confidence, and Lift – are explained using formulas, business interpretations, and Python implementations. Lift is highlighted as an important metric because it identifies meaningful recommendation patterns beyond simple popularity bias.

The Apriori Algorithm is demonstrated through candidate generation, pruning, support counting, and association rule generation. Flowcharts, tables, pseudocode, and Python implementations are used to explain the mining process step by step.

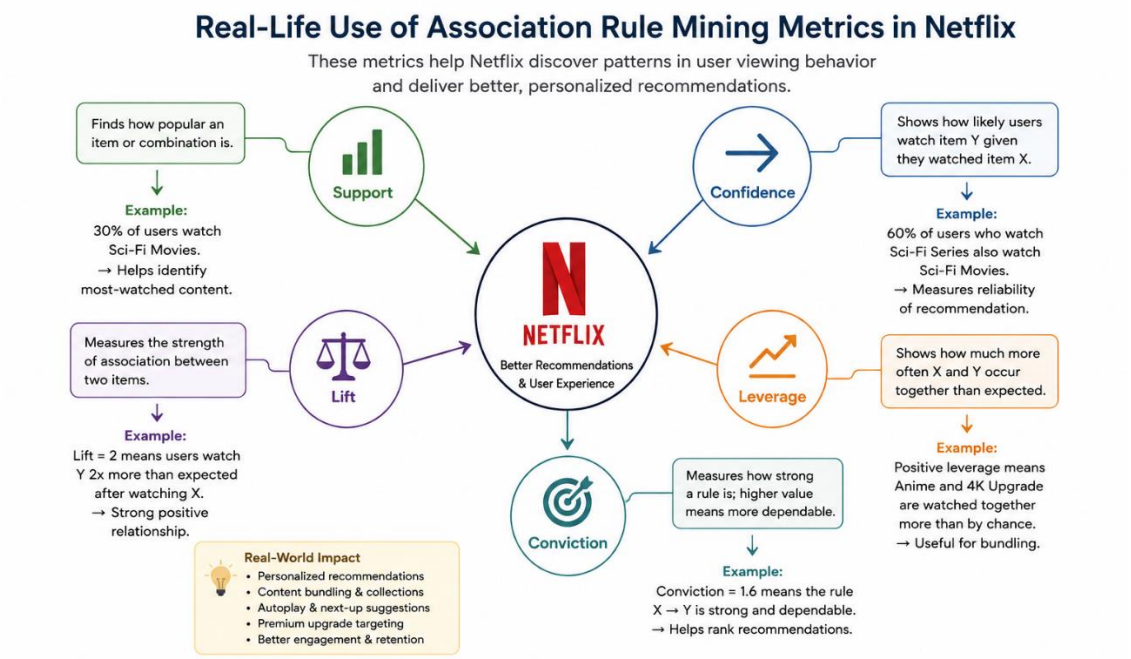
Streaming recommendation rules, gaming bundle recommendations, exam-style Apriori walkthroughs, FP-Growth optimization, and large-scale Netflix recommendation architecture are also covered. Technologies such as Apache Kafka, Apache Spark, Cassandra, and distributed FP-Growth systems are discussed to illustrate how modern streaming platforms process billions of watch events efficiently.

Overall, the content demonstrates how association rule mining techniques support personalization, recommendation ranking, content discovery, and viewer engagement in large-scale streaming platforms.

10.2 Formula Reference Table

Formula	Equation	Description
1. Support (X)	$\text{support}(X) = \frac{\text{count}(X)}{N}$	Proportion of transactions containing itemset X.
2. Support (X ∪ Y)	$\text{support}(X \cup Y) = \frac{\text{count}(X \cup Y)}{N}$	Proportion of transactions containing both X and Y.
3. Confidence (X → Y)	$\text{confidence}(X \to Y) = \frac{\text{support}(X \cup Y)}{\text{support}(X)}$	Likelihood of Y appearing in a transaction given X.
4. Lift (X → Y)	$\text{lift}(X \to Y) = \frac{\text{confidence}(X \to Y)}{\text{support}(Y)}$	Measures the strength of association between X and Y. • lift = 1 → no association • lift > 1 → positive association • lift < 1 → negative association
5. Leverage (X → Y)	$\text{leverage}(X \to Y) = \text{support}(X \cup Y) - \text{support}(X) \times \text{support}(Y)$	Difference between observed co-occurrence and expected co-occurrence if X and Y are independent.
6. Conviction (X → Y)	$\text{conviction}(X \to Y) = \frac{1 - \text{support}(Y)}{1 - \text{confidence}(X \to Y)}$	Measures the implication strength of the rule. • conviction > 1 → valid rule • higher value → stronger implication

Where: **count(·)** = Number of transactions containing the itemset | **N** = Total number of transactions in the database | **X, Y** = Itemsets



10.3 Apriori Algorithm

```
# ==========================================
# INPUT:
#     Transaction Database D
#     Minimum Support (min_support)
#     Minimum Confidence (min_confidence)#
# OUTPUT:
#     All Strong Association Rules
# ==========================================
```

```
from itertools import combinations

# ----------------------------------------------------------
# Sample Streaming Sessions
# Each session represents content watched together
# ----------------------------------------------------------

sessions = [
    ["Sci-Fi Movie", "Anime", "4K Upgrade"],
    ["Sci-Fi Movie", "Sci-Fi Series"],
    ["Anime", "Horror"],
    ["Sci-Fi Movie", "Documentary"],
    ["Sci-Fi Series", "Anime"],
    ["Sci-Fi Movie", "Anime"],
    ["Documentary", "Horror"],
    ["Sci-Fi Movie", "Sci-Fi Series", "Anime"],
    ["Anime", "4K Upgrade"],
    ["Sci-Fi Movie", "Horror"]
]


# ----------------------------------------------------------
# Function to calculate support count
# ----------------------------------------------------------

def count_support(itemset, sessions):

    count = 0

    for session in sessions:

        if itemset.issubset(session):

            count += 1

    return count


# ----------------------------------------------------------
# Apriori Algorithm
```

```
# ------------------------------------------------------

def apriori(sessions, min_support):

    items = set(item for s in sessions for item in s)

    frequent = {}
    current = []

    # ======================================================
    # PASS 1: Generate Frequent 1-Itemsets
    # ======================================================

    print("\n=== Generate Frequent 1-Itemsets ===\n")

    for item in items:

        itemset = frozenset([item])

        support = count_support(itemset, sessions)

        print(f"{set(itemset)} --> Support Count = {support}")

        if support >= min_support:

            frequent[itemset] = support
            current.append(itemset)

    print("\nFrequent 1-Itemsets:")

    for item in current:

        print(set(item))

    k = 2

    # ======================================================
    # Generate Larger Itemsets
    # ======================================================

    while current:

        print(f"\n=== Generate Larger Itemsets (k = {k}) ===\n")

        candidates = []

        # ------------------------------------------------
        # Candidate Generation
        # ------------------------------------------------
```

```
        for i, a in enumerate(current):

            for b in current[i + 1:]:

                union = a | b

                if len(union) == k:

                    subsets = [
                        frozenset(x)
                        for x in combinations(union, k - 1)
                    ]

                    if all(subset in frequent for subset in subsets):

                        if union not in candidates:

                            candidates.append(union)

        print("Generated Candidates:")

        for candidate in candidates:

            print(set(candidate))

        current = []

        # ------------------------------------------------
        # Support Counting
        # ------------------------------------------------

        print("\n=== Support Counting ===\n")

        for candidate in candidates:

            support = count_support(candidate, sessions)

            print(
                f"{set(candidate)} --> Support Count = {support}"
            )

            if support >= min_support:

                frequent[candidate] = support
                current.append(candidate)

        print("\nFrequent Itemsets in this Pass:")

        for item in current:
```

```
            print(set(item))

        k += 1

    return frequent


# ============================================================
# Run Apriori Algorithm
# ============================================================

print("\n===================================")
print(" Running Apriori Algorithm ")
print("===================================")

min_support = 2

frequent_itemsets = apriori(
    sessions,
    min_support
)


# ============================================================
# Display Final Results
# ============================================================

print("\n===================================")
print(" Final Frequent Itemsets ")
print("===================================\n")

for itemset, support in frequent_itemsets.items():

    print(
        f"{set(itemset)} --> Support Count = {support}"
    )
```

Following output will be generated

```
===================================

 Running Apriori Algorithm

===================================


=== Generate Frequent 1-Itemsets ===
```

```
{'Anime'} --> Support Count = 6
{'Sci-Fi Movie'} --> Support Count = 6
{'Documentary'} --> Support Count = 2
{'Sci-Fi Series'} --> Support Count = 3
{'Horror'} --> Support Count = 3
{'4K Upgrade'} --> Support Count = 2

Frequent 1-Itemsets:
{'Anime'}
{'Sci-Fi Movie'}
{'Documentary'}
{'Sci-Fi Series'}
{'Horror'}
{'4K Upgrade'}

=== Generate Larger Itemsets (k = 2) ===

Generated Candidates:
{'Sci-Fi Movie', 'Anime'}
{'Anime', 'Documentary'}
{'Sci-Fi Series', 'Anime'}
{'Horror', 'Anime'}
{'4K Upgrade', 'Anime'}
{'Sci-Fi Movie', 'Documentary'}
{'Sci-Fi Series', 'Sci-Fi Movie'}
{'Sci-Fi Movie', 'Horror'}
{'Sci-Fi Movie', '4K Upgrade'}
{'Sci-Fi Series', 'Documentary'}
{'Horror', 'Documentary'}
{'4K Upgrade', 'Documentary'}
```

```
{'Sci-Fi Series', 'Horror'}
{'Sci-Fi Series', '4K Upgrade'}
{'Horror', '4K Upgrade'}

=== Support Counting ===

{'Sci-Fi Movie', 'Anime'} --> Support Count = 3
{'Anime', 'Documentary'} --> Support Count = 0
{'Sci-Fi Series', 'Anime'} --> Support Count = 2
{'Horror', 'Anime'} --> Support Count = 1
{'4K Upgrade', 'Anime'} --> Support Count = 2
{'Sci-Fi Movie', 'Documentary'} --> Support Count = 1
{'Sci-Fi Series', 'Sci-Fi Movie'} --> Support Count = 2
{'Sci-Fi Movie', 'Horror'} --> Support Count = 1
{'Sci-Fi Movie', '4K Upgrade'} --> Support Count = 1
{'Sci-Fi Series', 'Documentary'} --> Support Count = 0
{'Horror', 'Documentary'} --> Support Count = 1
{'4K Upgrade', 'Documentary'} --> Support Count = 0
{'Sci-Fi Series', 'Horror'} --> Support Count = 0
{'Sci-Fi Series', '4K Upgrade'} --> Support Count = 0
{'Horror', '4K Upgrade'} --> Support Count = 0

Frequent Itemsets in this Pass:
{'Sci-Fi Movie', 'Anime'}
{'Sci-Fi Series', 'Anime'}
{'4K Upgrade', 'Anime'}
{'Sci-Fi Series', 'Sci-Fi Movie'}

=== Generate Larger Itemsets (k = 3) ===
```

```
Generated Candidates:
{'Sci-Fi Series', 'Sci-Fi Movie', 'Anime'}


=== Support Counting ===


{'Sci-Fi Series', 'Sci-Fi Movie', 'Anime'} --> Support Count = 1


Frequent Itemsets in this Pass:


==================================
 Final Frequent Itemsets
==================================


{'Anime'} --> Support Count = 6
{'Sci-Fi Movie'} --> Support Count = 6
{'Documentary'} --> Support Count = 2
{'Sci-Fi Series'} --> Support Count = 3
{'Horror'} --> Support Count = 3
{'4K Upgrade'} --> Support Count = 2
{'Sci-Fi Movie', 'Anime'} --> Support Count = 3
{'Sci-Fi Series', 'Anime'} --> Support Count = 2
{'4K Upgrade', 'Anime'} --> Support Count = 2
{'Sci-Fi Series', 'Sci-Fi Movie'} --> Support Count = 2
```

```
# ============================================
# Flow:
# Start
#   → Generate 1-itemsets
#   → Join candidates
#   → Prune (Apriori Principle)
#   → Scan Database
#   → Repeat
#   → Generate Rules
# ============================================
```

Today A Reader,
Tomorrow A Leader

Dezyne École College

www.dezyneecole.com

THANK YOU

Submitted By:- Navneet singh rawat

Department:- Bachelor of Computer Applications

Semester:- 6th

Session:- 2025-2026